# UE23CS352B - Object Oriented Analysis & Design

## Mini Project Report

## Title: Path Planning Sandbox

***Submitted by:***

***Team Members***

| NAME | SRN |
|---|---|
| Uday Gopan | PES2UG23CS659 |
| Unnathi Baskar | PES2UG23CS663 |
| Vishnu Prasath R S | PES2UG23CS703 |
| Shreyas Grama Srivathsan | PES2UG23CS901 |

6 - K

**Ms Vanitha A N**

**January - May 2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
FACULTY OF ENGINEERING
**PES UNIVERSITY**
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

**Problem Statement:**

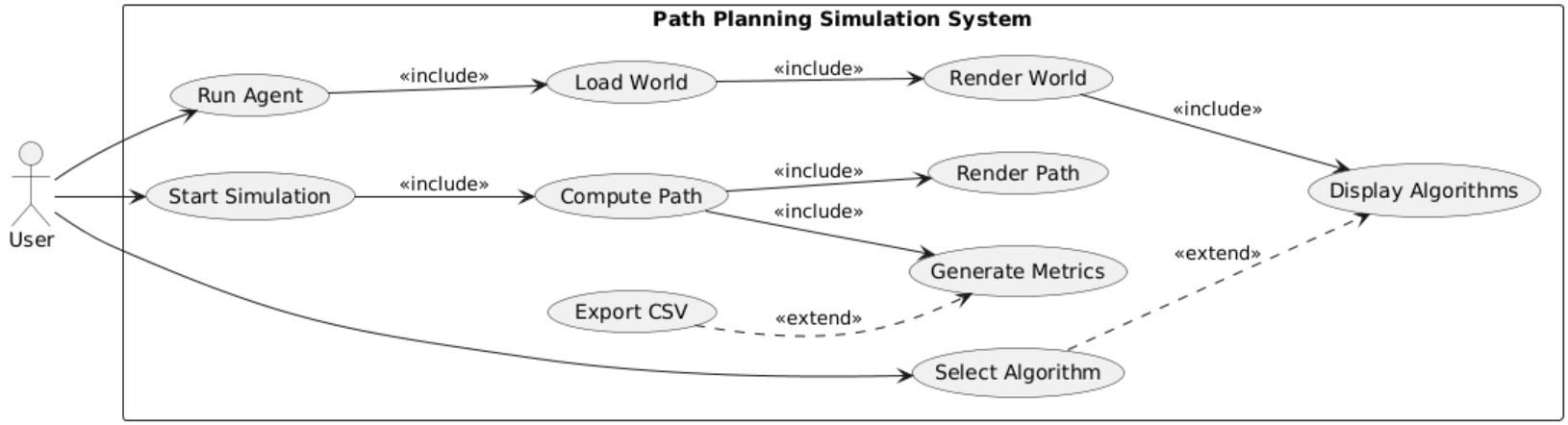
A modular, extensible system is needed to test, compare, and visualize different pathfinding algorithms across diverse map representations (2D grids and real-world geographic networks). The system must support dynamic algorithm selection, persistent execution history, and performance metrics tracking while maintaining clean separation of concerns through object-oriented design principles.

**Key Features:**

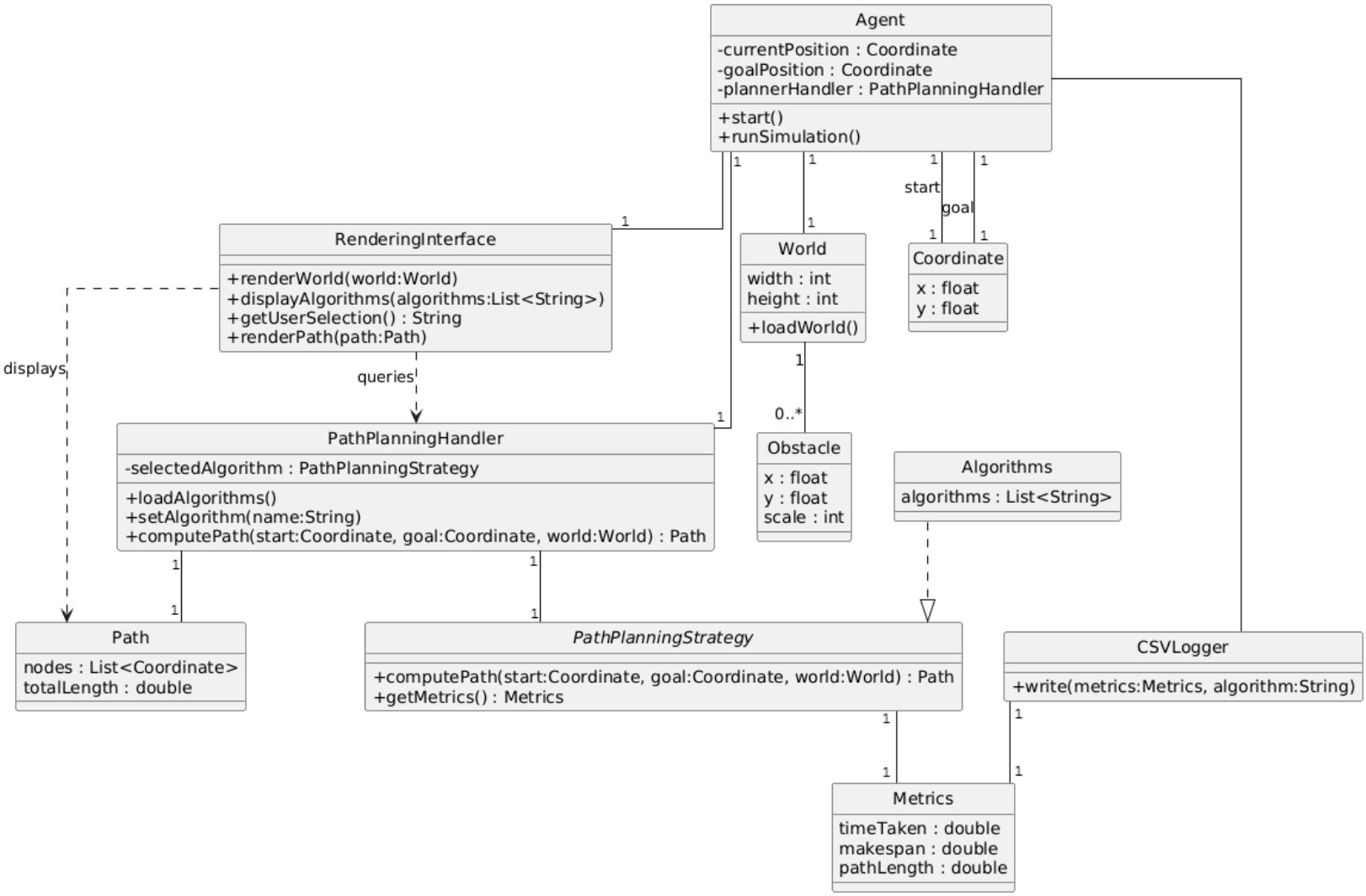
- Multiple pathfinding algorithms (Dijkstra's shortest path, Random Traversal baseline)
- Dual map support: JSON-based 2D grids and OpenStreetMap (OSM) XML networks
- Interactive web UI for coordinate input, algorithm selection, and result visualization
- H2 relational database for persistent execution history and performance logging
- Performance metrics tracking (execution time, path length, nodes expanded, success status)
- GeoJSON export for geographic pathfinding results
- Animated HTML visualization of traversed paths

Models:

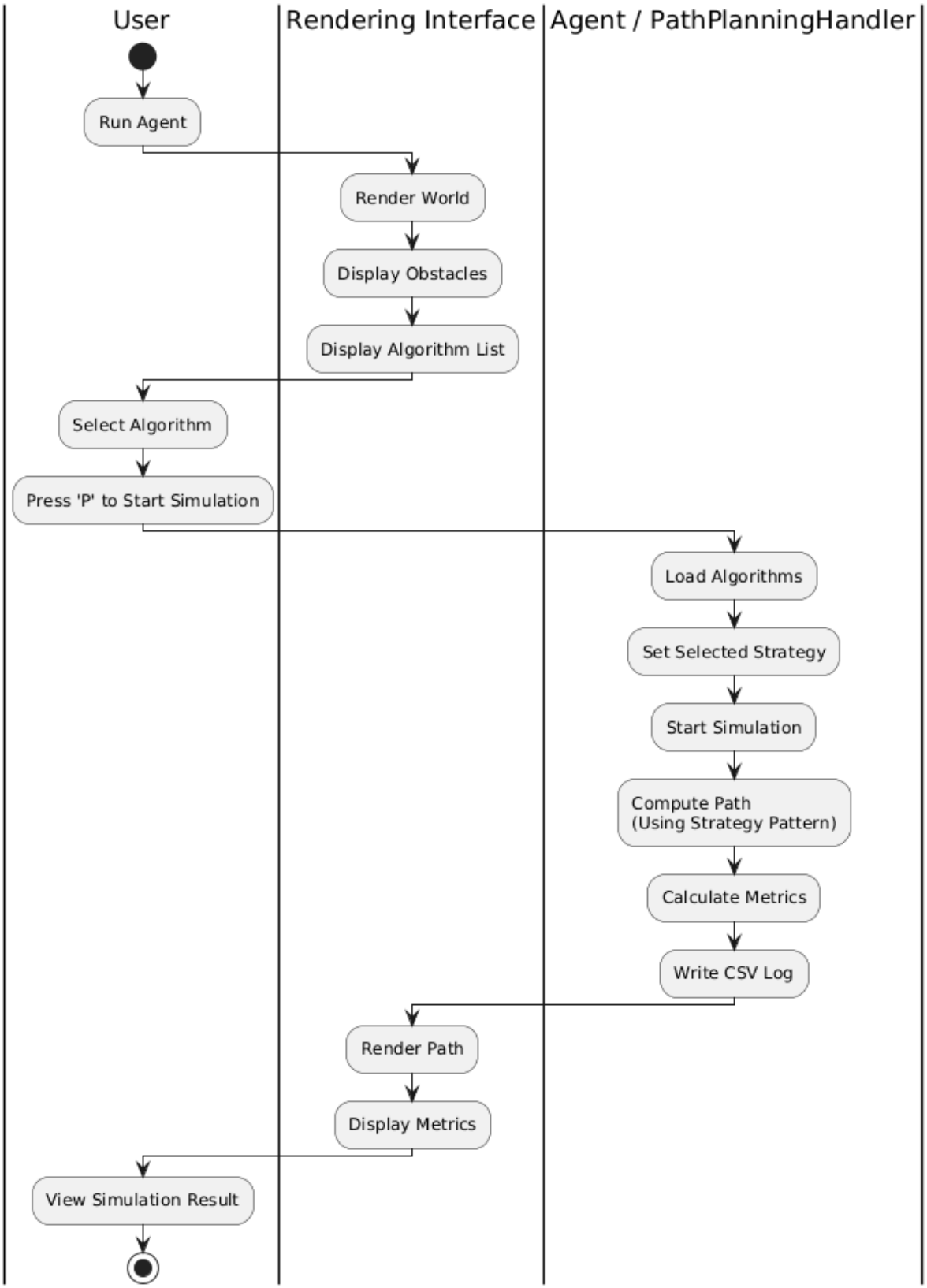
Use Case Diagram:



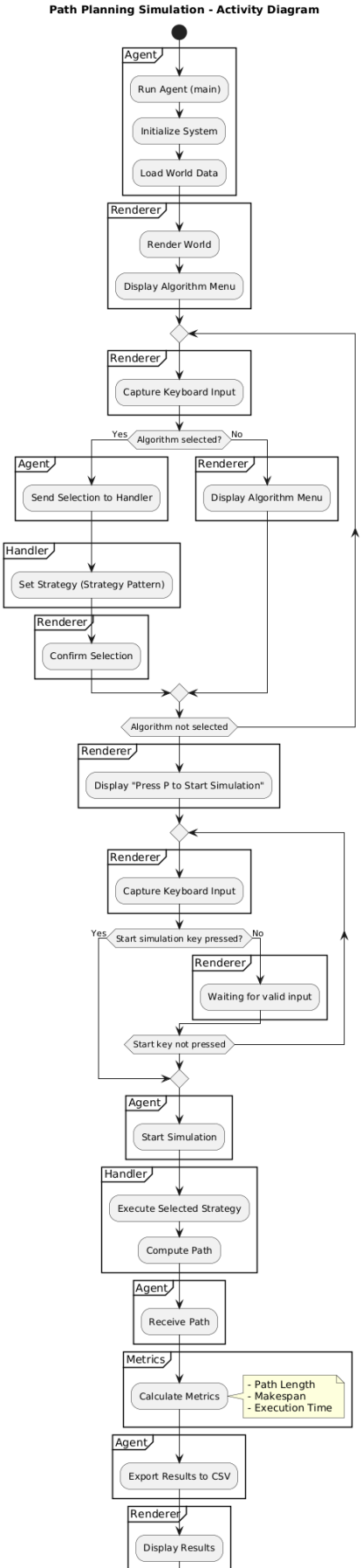
## Class Diagram:



## State Diagram:



# Activity Diagrams:



## Design Principles, and Design Patterns:

**MVC Architecture used?**
**Yes**

**Design Principles**
Separation of Concerns
The project is divided into 7 independent Maven modules, each handling a distinct responsibility:

- Core domain models are isolated in the core module
- Algorithm implementations reside in planners
- Input parsing logic is encapsulated in io
- Visualization is handled separately in rendering
- Metrics collection is independent in metrics
- This modularity prevents coupling and enables testing each layer independently

**Single Responsibility Principle (SRP)**
Each class has one reason to change:

- Coordinate - Manages position data only
- DijkstraPlanner - Implements Dijkstra algorithm only
- GridReader - Parses JSON grids only
- Renderer - Generates output visualizations only
- PathService - Orchestrates the workflow only

**Dependency Inversion**
The application depends on abstractions rather than concrete implementations:

- PathPlanner interface abstracts algorithm behavior
- GraphBuilder interface abstracts parsing logic
- Spring dependency injection inverts the dependency hierarchy, allowing loose coupling

**Open/Closed Principle (OCP)**
The system is open for extension but closed for modification:

- New pathfinding algorithms can be added by implementing PathPlanner without modifying existing code
- New map formats can be supported by creating new GraphBuilder implementations
- The core PathService remains unchanged when adding new planners

**Design Patterns**

1. Strategy Pattern (Behavioral)

Usage: PathPlanner interface with multiple implementations

- PathPlanner defines the interface for all pathfinding strategies
- DijkstraPlanner and RandomTraversalPlanner are concrete strategies
- PathService selects the appropriate strategy at runtime based on user input
- Allows runtime algorithm swapping without modifying service logic

2. Factory Pattern (Creational)

Usage: Map parsing and graph construction

- GridReader factory encapsulates JSON parsing complexity and returns AdjacencyGraph instances
- OsmGraphBuilder factory encapsulates OSM XML parsing complexity and returns AdjacencyGraph instances
- GraphBuilder interface abstracts the creation process
- Centralizes complex object creation logic and makes it maintainable

3. Adapter Pattern (Structural)

Usage: Coordinate abstraction for dual mapping systems

- Coordinate class acts as a universal adapter between two incompatible representations:
  - Cartesian integer coordinates (X, Y) for 2D grids
  - Double-precision Latitude/Longitude for geographic maps
- The core pathfinding engine remains agnostic to the underlying coordinate system
- Enables seamless switching between grid and OSM without algorithm modification

4. Singleton Pattern (Creational)

Usage: Spring Framework dependency injection

- PathService, Renderer, and database repositories are automatically managed as singletons by Spring
- Ensures single instances are shared across the entire application context
- Reduces memory overhead and maintains consistent state for shared resources

5. Repository Pattern (Behavioral)

Usage: Data persistence abstraction

- Spring JPA Repository interfaces abstract database operations
- PathExecutionLog entity is persisted through repository without exposing SQL queries
- Enables easy switching between database implementations

6. MVC Architecture

Usage: Complete separation of presentation, business logic, and data layers

- Model: Domain entities (Grid, Coordinate, PathResult), JPA entities

(PathExecutionLog)
- View: Thymeleaf templates (index.html, history.html, result.html)
- Controller: PathController receives HTTP requests and coordinates between services and views
- Clean routing: user input → controller → service → database/algorithms → view rendering

**Github link to the Codebase: (repository should be public)**

https://github.com/venator158/SandPP

## Screenshots

UI:

### SandPP - Path Planning Engine

#### Execute New Path

Map Input File (e.g. examples/grid1.json or examples/neighbour.osm)

examples/grid1.json

Start Coordinate (x,y or lat,lon)

0,0

Goal Coordinate (x,y or lat,lon)

4,4

Planner Algorithm

Dijkstra's Algorithm

Allow Diagonals (Grid Only)

Find Path

#### About SandPP

SandPP is an interactive path planning sandbox that allows you to visualize and evaluate different routing algorithms on Cartesian grids and OpenStreetMap data.

View Execution History

---

### Path Execution Results

| Metric | Value |
|---|---|
| ID | 4 |
| Algorithm | dijkstra |
| Map | examples/neighbour.osm |
| Start | 12.975,77.670 |
| Goal | 12.978,77.676 |
| Execution Time | 1 ms |
| Nodes Expanded | 343 |
| Path Length Steps | 40 |
| Actual Distance | 0.008045564638526036 |

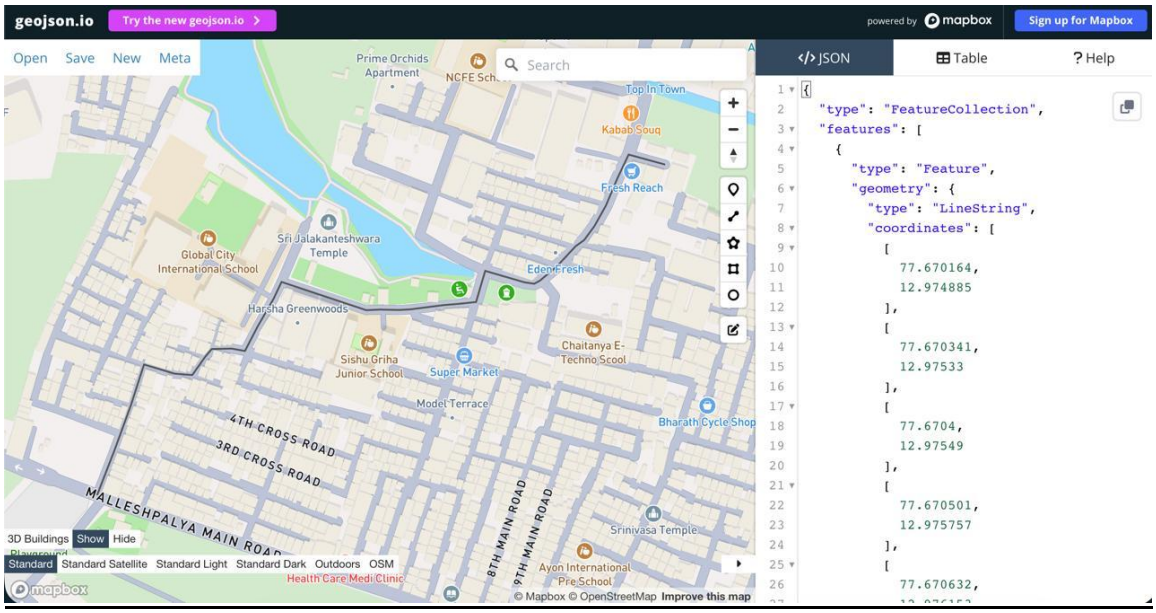
**Execution logged at:** 2026-04-20T12:08:05.355610

**View Rendering:**

Open Path in Geojson.io

Back to Menu | View Full History

---




## Path Execution Results

| Metric | Value |
|---|---|
| ID | 5 |
| Algorithm | dijkstra |
| Map | examples/grid1.json |
| Start | 0,0 |
| Goal | 4,4 |
| Execution Time | 0 ms |
| Nodes Expanded | 19 |
| Path Length Steps | 9 |
| Actual Distance | 8.0 |

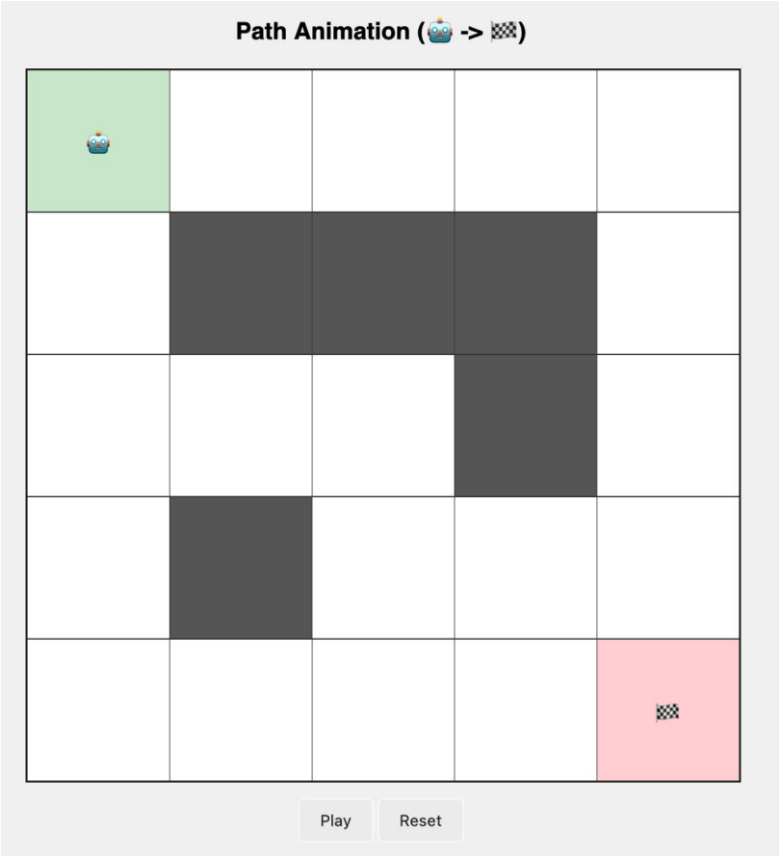
**Execution logged at:** 2026-04-20T12:11:13.119065

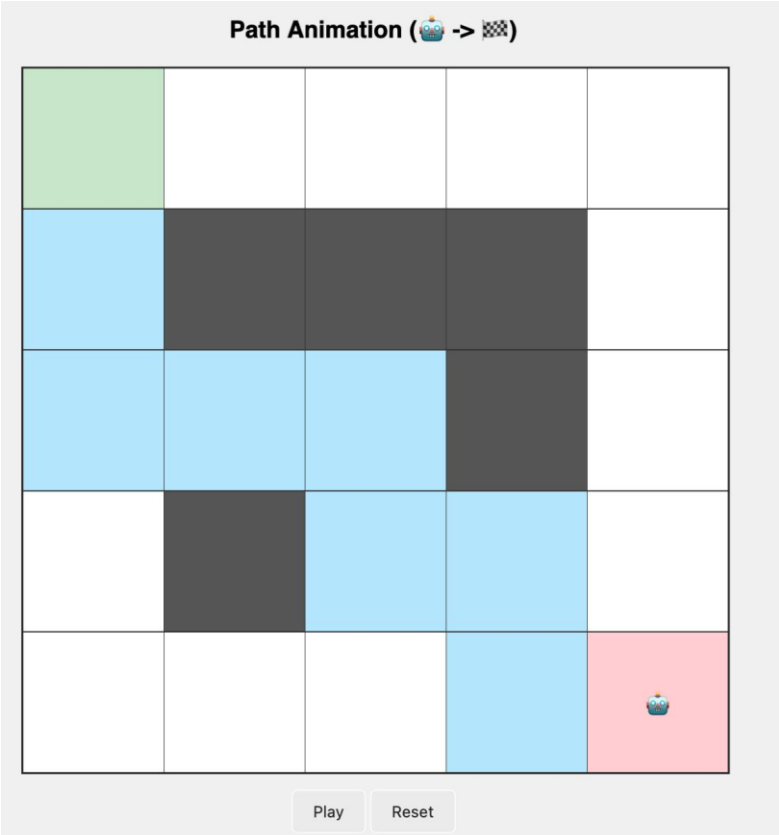
**View Rendering:**

The grid visualization was output directly to your local `output/index.html` folder.

For Web integration of Canvas rendering, this would require hosting the grid JSON as an API endpoint. But for now, you can open that file on your local machine.

Back to Menu | View Full History


Path Animation (🤖 -> 🏁)
Play
Reset


Path Animation (🤖 -> 🏁)
Play
Reset

## Pathfinding Execution History

Back to Menu

| ID | Run Date | Map | Algorithm | Start → Goal | Length | Exec Time (ms) | Nodes Expanded | GeoURL |
|---|---|---|---|---|---|---|---|---|
| 1 | 2026-04-20T11:32:12.310311 | examples/grid1.json | dijkstra | 0,0->4,4 | 9 | 2 | 19 | Local |
| 2 | 2026-04-20T11:33:21.606949 | examples/grid1.json | dijkstra | 0,0->4,4 | 9 | 0 | 19 | Local |

Individual contributions of the team members:

| **Name** | **Module worked on** |
|---|---|
| **Unnathi Baskar** | Domain Analysis & Map/Environment Modeling |
| **Uday Gopan** | Algorithm Selection & Planning Execution |
| **Shreyas GS** | Result Representation, Visualization & Metrics |
| **Vishnu Prasath RS** | Persistence, Experiment Management & Reproducibility |